Advanced Transport Phenomena

Practical session 2 - 2D transport and iterative Poisson solvers

Python edition

Alberto Cuoci and Adrien Guillonneau

2026

The point of this practical is to extend the 1D explicit Euler scheme from the first practical to two dimensions, and to introduce iterative methods (Jacobi, Gauss-Seidel, SOR) for solving the Poisson equation.

Contents

1 Python notes for this practical **2**

2 Advection-diffusion equation in 2D **2**
- 2.1 Case description . . . 2
- 2.2 Numerical setup . . . 2
- 2.3 Building numerical scheme . . . 3
- 2.4 Explicit Euler . . . 3
- 2.5 Bringing it together . . . 3
- 2.6 Graphical output . . . 4

3 Poisson equation in 2D **5**
- 3.1 Case description . . . 5
- 3.2 Numerical setup . . . 5
- 3.3 Iteration loop . . . 6
- 3.4 Graphical output . . . 6

1 Python notes for this practical

A NumPy 2D array is created with a shape tuple, `np.zeros((nx, ny))`, and indexed exactly like a MATLAB matrix, `f[i, j]` in place of `f(i,j)`, with the usual difference that indices start at 0.

The boundary conditions below select a segment of the domain using MATLAB expressions like `ny*1/3 : ny*2/3+1`. The general rule for translating a MATLAB range `a(s:e)` into Python is `a[s-1:e]`: shift the start by one, and leave the stop as it is.

2 Advection-diffusion equation in 2D

2.1 Case description

The advection-diffusion equation in 2D, over a rectangular domain of size $L_x \times L_y$:

$$\frac{\partial f}{\partial t} + u\frac{\partial f}{\partial x} + v\frac{\partial f}{\partial y} = \Gamma\left(\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}\right) \tag{1}$$

Along the north ($y = L_y$), south ($y = 0$), and west ($x = 0$) sides, zero gradient (Neumann) boundary conditions are prescribed:

$$\frac{\partial f}{\partial n} = 0 \tag{2}$$

Along the east side ($x = L_x$), a Dirichlet condition is prescribed over the central third of the boundary:

$$f\left(x = L_x;\ y \in \left[\frac{L_y}{3}, \frac{2L_y}{3}\right]\right) = f_{inlet} \qquad f\left(x = L_x;\ y \in \left[0, \frac{L_y}{3}\right] \cup \left[\frac{2L_y}{3}, L_y\right]\right) = 0 \tag{3}$$

Explicit (forward) Euler in time, centered 2nd order differences in space:

$$\frac{f_{i,j}^{n+1} - f_{i,j}^{n}}{\Delta t} + u\,\frac{f_{i+1,j}^{n} - f_{i-1,j}^{n}}{2h_x} + v\,\frac{f_{i,j+1}^{n} - f_{i,j-1}^{n}}{2h_y} = \Gamma\,\frac{f_{i+1,j}^{n} - 2f_{i,j}^{n} + f_{i-1,j}^{n}}{h_x^2} + \Gamma\,\frac{f_{i,j+1}^{n} - 2f_{i,j}^{n} + f_{i,j-1}^{n}}{h_y^2} \tag{4}$$

2.2 Numerical setup

A regular 2D squared grid is considered ($L_x = L_y$, $n_x = n_y$), with $u = -1$ and $v = 0$, a uniform velocity field from east to west.

```
import numpy as np
import matplotlib.pyplot as plt

nx = 33        # number of grid points along x
ny = 33        # number of grid points along y
nstep = 150    # number of time steps
lengthx = 2.0  # domain length along x [m]
lengthy = 2.0  # domain length along y [m]
D = 0.025      # diffusion coefficient [m2/s] GAMMA
u = -1.0       # velocity along x [m/s]
v = 0.0        # velocity along y [m/s]
fin = 1.0      # inlet value of f

hx = lengthx / (nx - 1) # grid step along x [m]
hy = lengthy / (ny - 1) # grid step along y [m]
```

```
sigma = 0.75
dt_diff = 0.25 * min(hx**2, hy**2) / D
dt_conv = 4 * D / (u**2 + v**2)
dt = sigma * min(dt_diff, dt_conv)
```

2.3 Building numerical scheme

The solution array is pre-allocated, and the inlet Dirichlet condition is fixed on the east side.

```
f = np.zeros((nx, ny))

j1 = ny * 1 // 3
j2 = ny * 2 // 3 + 1
f[nx - 1, j1 - 1:j2] = fin
```

2.4 Explicit Euler

The interior update follows the same forward Euler pattern as the 1D scheme, applied along both directions at once.

```
def explicit_step_2d(f_old, nx, ny, hx, hy, dt, u, v, D):
    '''One explicit Euler step for the 2D advection-diffusion equation.'''
    f_new = f_old.copy()

    for i in range(1, nx - 1):
        for j in range(1, ny - 1):
            f_new[i, j] = (f_old[i, j]
                           - (0.5 * dt * u / hx) * (f_old[i+1, j] - f_old[i-1, j])
                           - (0.5 * dt * v / hy) * (f_old[i, j+1] - f_old[i, j-1])
                           + (D * dt / hx**2) * (f_old[i+1, j] - 2*f_old[i, j] + f_old[i
                               -1, j])
                           + (D * dt / hy**2) * (f_old[i, j+1] - 2*f_old[i, j] + f_old[i
                               , j-1]))

    # Neumann boundary conditions (south, north, west): left as an exercise
    # f_new[:, 0] = ...
    # f_new[:, -1] = ...
    # f_new[0, :] = ...

    return f_new
```

Three of the four edges need a zero-gradient (Neumann) condition, meaning the boundary value equals its nearest interior neighbor from the previous step. The east edge needs nothing here, it is a steady Dirichlet condition already fixed once, and a boundary condition that never changes in time never needs to be reapplied.

Complete the three commented lines above, then check: with $u = 0$ and $v = 0$ (no advection), the field should relax so that every column and row runs flat near the south, north, and west edges.

2.5 Bringing it together

```
def run_advection_diffusion_2d(nx, ny, nstep, lengthx, lengthy, D, u, v, fin, verbose
    =True):
    hx = lengthx / (nx - 1)
    hy = lengthy / (ny - 1)
```

```
    sigma = 0.75
    dt_diff = 0.25 * min(hx**2, hy**2) / D
    dt_conv = 4 * D / (u**2 + v**2)
    dt = sigma * min(dt_diff, dt_conv)
    if verbose:
        print(f"dt = {dt:.5f}")

    f = np.zeros((nx, ny))
    j1 = ny * 1 // 3
    j2 = ny * 2 // 3 + 1
    f[nx - 1, j1 - 1:j2] = fin

    f_history, t_history = [], []

    t = 0.0
    for m in range(nstep):
        f_history.append(f.copy())
        t_history.append(t)

        f = explicit_step_2d(f, nx, ny, hx, hy, dt, u, v, D)
        t = t + dt
        if verbose and m % 25 == 0:
            print(f"time={t:.4f}")

    return {"f_history": f_history, "t_history": t_history, "hx": hx, "hy": hy}

result = run_advection_diffusion_2d(nx=33, ny=33, nstep=150, lengthx=2.0, lengthy
    =2.0, D=0.025, u=-1.0, v=0.0, fin=1.0)
```

Running this cell without completing the Neumann boundary conditions will not raise an error, but the south, north, and west edges will silently stay at whatever value they last held, instead of following the interior field. An incomplete boundary condition does not always crash, sometimes it just gives a quietly wrong answer.

2.6 Graphical output

For a 2D field, a color map (`pcolormesh`) is simpler to animate than a 3D surface and carries the same information. The full animation code is in the companion notebook.

Exercises

1. Complete the Neumann boundary conditions in the explicit Euler section, then check them with the flat-field test described there.

2. Analyze the effect of the diffusion coefficient on the solution.

3. Calculate the solution corresponding to the following new boundary conditions along the east side:

$$f\left(x = L_x;\ y \in \left[\frac{L_y}{5}, \frac{2L_y}{5}\right] \cup \left[\frac{3L_y}{5}, \frac{4L_y}{5}\right]\right) = f_{inlet}$$

$$f\left(x = L_x;\ y \in \left[0, \frac{L_y}{5}\right] \cup \left[\frac{2L_y}{5}, \frac{3L_y}{5}\right] \cup \left[\frac{4L_y}{5}, L_y\right]\right) = 0$$

4. Implement a FDS and a BDS for the advection term in this 2D case and compare the results.

3 Poisson equation in 2D

3.1 Case description

The Poisson equation on a 2D rectangular domain, solved with the Successive Over-Relaxation method:

$$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = S(x, y) \tag{5}$$

Using standard finite differences and, for equal grid steps ($h_x = h_y = h$):

$$f_{i,j} = \frac{1}{4}\left(f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - h^2 S_{i,j}\right) \tag{6}$$

Jacobi: solve for $f_{i,j}$ using only old values on the right hand side. Simple, but many iterations are needed. The residual, which should be zero at steady state, measures how far a candidate solution is from satisfying the equation:

$$R^{\alpha}_{i,j} = \frac{f^{\alpha}_{i+1,j} - 2f^{\alpha}_{i,j} + f^{\alpha}_{i-1,j}}{h_x^2} + \frac{f^{\alpha}_{i,j+1} - 2f^{\alpha}_{i,j} + f^{\alpha}_{i,j-1}}{h_y^2} - S^{\alpha}_{i,j} \tag{7}$$

Gauss-Seidel: the same idea, but using new values as soon as they become available, converges markedly faster.

SOR: pushes the Gauss-Seidel correction further at each step, by a factor β:

$$f^{\alpha+1}_{i,j} = \left[\frac{1}{2(h_x^2 + h_y^2)}\left(h_y^2 f^{\alpha}_{i+1,j} + h_y^2 f^{\alpha+1}_{i-1,j} + h_x^2 f^{\alpha}_{i,j+1} + h_x^2 f^{\alpha+1}_{i,j-1} - h_x^2 h_y^2 S^{\alpha}_{i,j}\right)\right]\beta + f^{\alpha}_{i,j}(1-\beta) \tag{8}$$

$\beta = 1$ recovers Gauss-Seidel exactly. $1 < \beta < 2$ is required for SOR to accelerate convergence; $\beta = 1.5$ is usually a good starting value.

3.2 Numerical setup

```
lengthx = 2.0
lengthy = 2.0
nx = 39
ny = 39
max_iterations = 5000
beta = 1.0           # SOR coefficient (1 means Gauss-Seidel)
threshold = 0.001

hx = lengthx / (nx - 1)
hy = lengthy / (ny - 1)

f = np.zeros((nx, ny))
S = np.zeros((nx, ny))

j1 = ny * 1 // 3
j2 = ny * 2 // 3
f[0, j1 - 1:j2] = 1.0 # Dirichlet condition, west side
```

3.3 Iteration loop

The SOR formula above updates every interior point, using the most recent values available, and the residual measures how close the current field is to a solution. Both are left to complete: the formula for `f[i, j]` is a direct translation of the equation above, and the residual follows the same pattern as the update, but with the original, undivided Laplacian.

```
def run_poisson_2d_sor(lengthx, lengthy, nx, ny, max_iterations, beta, threshold,
    verbose=True):
    hx = lengthx / (nx - 1)
    hy = lengthy / (ny - 1)

    f = np.zeros((nx, ny))
    S = np.zeros((nx, ny))
    j1 = ny * 1 // 3
    j2 = ny * 2 // 3
    f[0, j1 - 1:j2] = 1.0

    tot_res = []

    for l in range(max_iterations):

        for i in range(1, nx - 1):
            for j in range(1, ny - 1):
                pass
                # f[i, j] = ... # SOR update

        res = 0.0
        for i in range(1, nx - 1):
            for j in range(1, ny - 1):
                pass
                # res += ...     # residual at (i, j)

        raise NotImplementedError("complete the SOR update and the residual above,
            then remove this line")

        tot_res.append(res / ((nx-2) * (ny-2)))
        if verbose and l % 25 == 0:
            print(f"Iteration: {l}, Residual: {tot_res[-1]:.6e}")

        if tot_res[-1] < threshold:
            break

    return {"f": f, "tot_res": tot_res, "hx": hx, "hy": hy}

result_poisson = run_poisson_2d_sor(lengthx=2.0, lengthy=2.0, nx=39, ny=39,
                                    max_iterations=5000, beta=1.0, threshold=0.001)
```

3.4 Graphical output

```
xaxis = np.linspace(0, lengthx, nx)
yaxis = np.linspace(0, lengthy, ny)

plt.figure()
plt.pcolormesh(xaxis, yaxis, result_poisson["f"].T, shading="auto", cmap="jet")
plt.colorbar(label="f value")
plt.xlabel("x-axis [m]")
```

```
plt.ylabel("y-axis [m]")
plt.show()

plt.figure()
plt.semilogy(range(1, len(result_poisson["tot_res"])+1), result_poisson["tot_res"])
plt.xlabel("iteration")
plt.ylabel("residual")
plt.show()
```

Exercises

1. Identify the optimal value of β, the value which minimizes the number of iterations.
2. Demonstrate that the numerical discretization adopted is 2nd order accurate in space.
3. Solve a new case, in which the boundary conditions are $f = 0$ everywhere (on the west side too) and the source term has the following expression:

$$S(x, y) = (x - 1)^2 + (y - 1)^2$$